# XGBoost

## eXtreme Gradient

## Boosting

*Boosting · Gradient Descent on Trees · Regularization · Kaggle Champion*

# What We'll Cover Today

## 01
### From RF to Boosting
Why sequential beats parallel

## 02
### How XGBoost Works
Gradient boosting + regularization

## 03
### The Math Behind XGBoost
Objective function & Taylor expansion

## 04
### Key Hyperparameters
n_estimators, learning_rate, max_depth…

## 05
### XGBoost in Python
xgboost library & scikit-learn API

## 06
### Pros, Cons & Use Cases
When to use XGBoost — and when not to

# From Random Forest to XGBoost — Two Different Strategies

| Random Forest — BAGGING | |
|---|---|
| Strategy | Parallel — train all trees independently |
| Sampling | Each tree on random bootstrap sample |
| Errors | Independent — reduce variance |
| Combine | Majority vote / average |
| Learns from | Data diversity |
| Main strength | Robust, hard to overfit |

| XGBoost — BOOSTING | |
|---|---|
| Strategy | Sequential — each tree fixes the last |
| Sampling | Whole dataset, learns from residuals |
| Errors | Corrected — reduce bias |
| Combine | Weighted sum of all trees |
| Learns from | Previous mistakes |
| Main strength | Extremely accurate, Kaggle champion |

# What is Boosting? The Core Intuition

**Real-Life Analogy:** A student takes a test, gets marked wrong on certain questions. They study ONLY the wrong answers, take the test again, get marked again — and repeat. Each round targets the previous mistakes.

**1**

**Train weak tree**

Fit a shallow decision tree (depth 3–6) on the training data

**2**

**Compute residuals**

Find where the model was wrong
Residual = $y - \hat{y}$
(the errors)

**3**

**Train next tree**

Fit a NEW tree on the RESIDUALS — targets the previous errors

**4**

**Add to ensemble**

New prediction = old prediction + $\alpha \times$ new tree
($\alpha$ = learning rate)

**5**

**Repeat N times**

After N rounds errors shrink predictions improve steadily

**Key formula:** $\hat{F}_m(x) = \hat{F}_{m-1}(x) + \eta \cdot h_m(x)$ where $\eta$ = learning rate, $h_m$ = new tree

# Boosting in Action — Step-by-Step Numerical Example

*Goal: predict house price. Initial prediction = mean of all prices = $200k*

| House | True Price ($k) | Round 1 Pred | Residual 1 | Round 2 Pred | Residual 2 | Final Pred |
|---|---|---|---|---|---|---|
| A | 150 | 200 | −50 | 155 | −5 | **155** |
| B | 250 | 200 | +50 | 245 | +5 | **245** |
| C | 300 | 200 | +100 | 290 | +10 | **290** |
| D | 100 | 200 | −100 | 110 | −10 | **110** |

### Round 1 (Tree 1)

All predict $200k (mean). Residuals show how wrong. Tree 1 learns to predict residuals.

### Round 2 (Tree 2)

Add Tree 2 predictions (× learning rate 0.1) to Round 1. Residuals get 10× smaller.

### After N rounds

Residuals approach zero. Each new tree corrects the remaining error of all previous trees.

# Gradient Boosting — Using Gradient Descent on Trees

**Key insight:** Fitting a tree on RESIDUALS is mathematically equivalent to performing gradient descent in function space — we're moving the prediction in the direction that reduces loss most.

## Step 1: Define Loss

For regression: MSE = $\frac{1}{2}\cdot(y - \hat{y})^2$

For classification: Log-loss = $-y\cdot\log(p) - (1-y)\cdot\log(1-p)$

The loss tells us how 'wrong' the current model is for each sample.

We want to MINIMIZE the total loss over the entire dataset.

## Step 2: Compute Gradients

Gradient = $\partial L/\partial\hat{y}$ (how loss changes with prediction)

For MSE: gradient = $-(y - \hat{y})$ = residual!

For log-loss: gradient = $\hat{y} - y$ (probability error)

These gradients are the TARGET values for the next tree to learn.

Negative gradient = direction of steepest loss decrease.

## Step 3: Fit Tree on Gradients

Train tree $h_t$ to predict the gradient values

Each leaf learns the optimal step in gradient direction

Update: $\hat{F}_t = \hat{F}_{t-1} + \eta \cdot h_t$

η (learning rate) controls step size — exactly like neural network gradient descent!

Any differentiable loss function works — regression, classification, ranking, custom objectives. This is XGBoost's superpower.

# XGBoost Objective — What Makes It 'Extreme'

*XGBoost adds regularization directly into the objective — this is what separates it from vanilla gradient boosting.*

$$\mathrm{Obj}(\theta) = \Sigma_i\, L(y_i, \hat{y}_i) + \Sigma_k\, \Omega(f_k)$$

*Loss term + Regularization term*

$$\Omega(f) = \gamma \cdot T + \tfrac{1}{2}\lambda \cdot \Sigma_j\, w_j^2$$

### $L(y_i, \hat{y}_i)$ — Loss Function

Measures how wrong each prediction is. MSE for regression, log-loss for classification. Standard from gradient boosting.

### $\Omega(f_k)$ — Regularization

Penalizes model complexity. XGBoost-exclusive. Prevents overfitting by discouraging too many leaves or large leaf weights.

### $\gamma \cdot T$ — Leaf penalty

$\gamma$ (gamma) penalizes the NUMBER of leaves T. Higher $\gamma$ → fewer splits → simpler trees. Set 0 for no pruning.

### $\tfrac{1}{2}\lambda \cdot \Sigma w_j^2$ — L2 on weights

$\lambda$ (lambda) penalizes LARGE leaf weight values $w_j$. Like L2/Ridge regularization in linear regression. Smooths predictions.

# Second-Order Approximation — Why XGBoost is Faster

*XGBoost uses a 2nd-order Taylor expansion of the loss — this gives a closed-form optimal leaf weight, skipping expensive iterations.*

$$L \approx \Sigma_i \left[ L(y_i, \hat{y}_i^{(t-1)}) + g_i \cdot f_t(x_i) + \tfrac{1}{2} \cdot h_i \cdot f_t(x_i)^2 \right] + \Omega(f_t)$$

*where $g_i = \partial L / \partial \hat{y}_i$ (1st derivative = gradient) and $h_i = \partial^2 L / \partial \hat{y}_i^2$ (2nd derivative = Hessian)*

## $g_i$ (Gradient)

First derivative of loss w.r.t. prediction.
For MSE: $g_i = \hat{y}_i - y_i$ (residual)
For log-loss: $g_i = p_i - y_i$
Tells us: in which DIRECTION to move $\hat{y}$

## $h_i$ (Hessian)

Second derivative of loss w.r.t. prediction.
For MSE: $h_i = 1$ (constant)
For log-loss: $h_i = p_i(1-p_i)$
Tells us: how FAST to move (curvature)

**Optimal leaf weight:**

$$w^*_j = -\Sigma_{i \in I_j} g_i \;/\; (\Sigma_{i \in I_j} h_i + \lambda)$$

**Why this matters:**

Traditional gradient boosting uses only $g_i$ (first order). XGBoost uses BOTH $g_i$ and $h_i$ — the Hessian acts like an adaptive learning rate, making convergence faster and more stable.

# Key Hyperparameters — Tuning XGBoost

| | | |
|---|---|---|
| `n_estimators` | default: 100 | Number of trees. More = better accuracy until overfitting. Combine with early_stopping_rounds. |
| `learning_rate` | default: 0.3 | η — shrinks each tree's contribution. Lower (0.01–0.1) + more trees = best combo. The most important param. |
| `max_depth` | default: 6 | Depth of each tree. XGBoost uses shallow trees (3–8). Deeper = more complex, higher overfit risk. |
| `subsample` | default: 1.0 | Fraction of training rows per tree. 0.8 means 80% random sample. Adds variance reduction like RF. |
| `colsample_bytree` | default: 1.0 | Fraction of features sampled per tree. Like Random Forest's max_features. 0.7–0.9 often optimal. |
| `gamma (min_split_loss)` | default: 0 | Minimum loss reduction to make a split. Higher γ → more conservative trees. Good for noisy data. |
| `reg_alpha (L1)` | default: 0 | L1 regularization on leaf weights. Produces sparse models (some weights = 0). Good for many features. |
| `reg_lambda (L2)` | default: 1 | L2 regularization on leaf weights. Always active by default. Controls weight magnitude. Start here. |

# Special XGBoost Features — Early Stopping & Tree Pruning

## Early Stopping

Problem: with 1000 trees, how many do we actually need?

Solution: monitor validation loss every round. Stop when it hasn't improved for N rounds.

```
model = xgb.XGBClassifier(
    n_estimators=1000,
    early_stopping_rounds=50
)
model.fit(X_tr, y_tr,
    eval_set=[(X_val, y_val)])
```

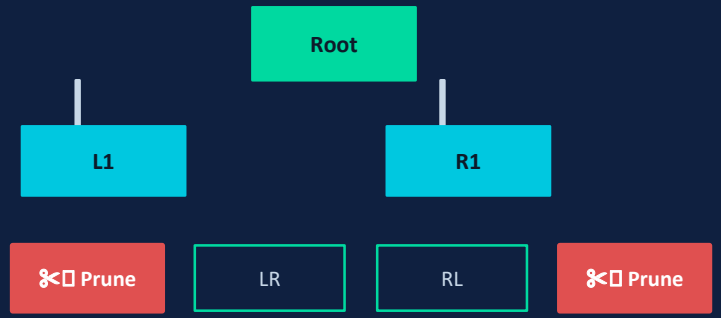
Trains with n_estimators=1000 but stops early if val loss plateaus for 50 rounds. Returns best_ntree_limit automatically.

Val loss vs. rounds → stops at ★ best

## Max-to-Root Pruning

Traditional CART trees: stop splitting if gain < threshold (greedy).

XGBoost: grow the full tree to max_depth FIRST, then prune back nodes with negative gain.




*Red nodes: negative gain after full expansion → pruned away*

Benefit: XGBoost can discover beneficial splits BELOW seemingly bad ones — something greedy CART misses completely.

# XGBoost in Python — Full Implementation

```python
import xgboost as xgb
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import accuracy_score, roc_auc_score
from sklearn.preprocessing import LabelEncoder

# 1. Load & split data
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=42)
X_tr, X_val, y_tr, y_val = train_test_split(X_tr, y_tr, test_size=0.1,
random_state=42)
# 2. Define model with key hyperparameters
model = xgb.XGBClassifier(
    n_estimators     = 1000,   # upper bound — early stopping decides
    learning_rate    = 0.05,   # low lr + many trees = best accuracy
    max_depth         = 6,     # shallow trees prevent overfitting
    subsample         = 0.8,   # row sampling like Random Forest
    colsample_bytree = 0.8,   # column sampling
    gamma             = 0.1,   # min gain to split
    reg_lambda        = 1.5,   # L2 regularization
    early_stopping_rounds = 50,
    eval_metric = 'logloss',  use_label_encoder=False,
    random_state = 42)

# 3. Train with validation monitoring
model.fit(X_tr, y_tr, eval_set=[(X_val, y_val)], verbose=100)

# 4. Evaluate
y_pred = model.predict(X_te)
y_prob = model.predict_proba(X_te)[:, 1]
print(f'Accuracy: {accuracy_score(y_te, y_pred):.4f}')
print(f'ROC-AUC : {roc_auc_score(y_te, y_prob):.4f}')

# 5. Feature importance
xgb.plot_importance(model, max_num_features=10)
```

### Installation

pip install xgboost
Works with scikit-learn API. Also supports DMatrix format for max speed.

### eval_set

Pass validation data here. XGBoost tracks loss each round and triggers early stopping automatically.

### predict_proba()

Returns probability for each class. Use for ROC-AUC, threshold tuning, calibration.

### plot_importance()

Built-in feature importance plot. Types: 'weight' (split count), 'gain' (avg improvement), 'cover'.

```python
Also works with DMatrix:
dmatrix = xgb.DMatrix(X, label=y)
```

# XGBoost vs Random Forest — Head-to-Head

| | XGBoost | Random Forest |
|---|---|---|
| Learning strategy | Sequential boosting | Parallel bagging |
| Error reduction | Bias ↓ (corrects mistakes) | Variance ↓ (averages diversity) |
| Training speed | Slower (sequential) | Faster (parallel) |
| Accuracy (typical) | Higher ✓ | Good but lower |
| Overfitting risk | Higher if mis-tuned | Lower (more robust) |
| Hyperparameter tuning | More params to tune | Fewer, more forgiving |
| Missing data | Handles natively ✓ | Must impute |
| Feature importance | gain / weight / cover types | impurity-based |
| Regularization | Built-in L1 + L2 + γ ✓ | None built-in |
| Best use case | Competitions, tabular data wins | Quick robust baseline |

# XGBoost — Advantages & Disadvantages

## ADVANTAGES

### ✓ State-of-the-Art Accuracy
Wins most tabular data competitions. Often 5–15% better than Random Forest with proper tuning.

### ✓ Built-in Regularization
L1 (sparsity), L2 (smoothness), γ (leaf penalty) built into the objective. Resistant to overfitting.

### ✓ Handles Missing Data
Learns the optimal default direction for missing values — no imputation needed.

### ✓ Speed & Parallelism
Column-parallel split finding, cache-optimized, GPU support. Faster than you'd expect for sequential.

### ✓ Flexible Loss Functions
Regression, binary/multiclass classification, ranking, custom objectives. One framework for everything.

## DISADVANTAGES

### ✗ Many Hyperparameters
10+ important params. Requires careful tuning or auto-ml. More complex than Random Forest.

### ✗ Sensitive to Outliers
Boosting amplifies outliers across rounds. Random Forest is more robust to noisy labels/features.

### ✗ Slower Training than RF
Sequential nature can't be fully parallelized. RF is faster to train, especially with few trees.

### ✗ Not Ideal for Small Data
With <500 samples, XGBoost can overfit even with regularization. Prefer simpler models.

### ✗ Not for Unstructured Data
Images, text, audio — use CNNs/Transformers. XGBoost shines on tabular data only.

# Decision Guide — When to Use XGBoost?

| USE XGBoost when: | AVOID XGBoost when: |
| --- | --- |
| ✓ You have structured/tabular data | ✗ Images, audio, or text data → use DL |
| ✓ Accuracy is the top priority | ✗ Very small datasets (<500 samples) |
| ✓ Dataset is 1k–10M+ rows | ✗ Need a fully interpretable model |
| ✓ Missing values present in data | ✗ Real-time inference <1ms required |
| ✓ You're entering a Kaggle competition | ✗ Time series with temporal patterns → LSTM |
| ✓ Need feature importance insights | ✗ Data is very clean, simple & linear |

**Rule of Thumb:** Try Random Forest first (fast baseline), then XGBoost. If XGBoost gains >2% — invest time in tuning. If not — RF is good enough and simpler to maintain.

# Session Summary — Key Takeaways

## 01

### Boosting = Sequential learning

Each tree corrects the mistakes of all previous trees. Targets residuals / gradients — unlike parallel bagging.

## 02

### Gradient in function space

Gradient boosting applies gradient descent — but on TREES instead of parameters. $g_i$ and $h_i$ guide each new tree.

## 03

### Objective = Loss + Regularization

Obj = $\Sigma\, L(y_i, \hat{y}_i) + \Sigma\, \Omega(f_t)$. Built-in L1, L2, and γ prevent overfitting directly at the math level.

## 04

### Key params: lr + n_estimators

Low learning_rate (0.01–0.1) + high n_estimators + early stopping = best XGBoost recipe.

## 05

### Handles missing data natively

Learns optimal default split direction for NaN values. No imputation needed — unique XGBoost advantage.

# Great work!

You now understand XGBoost.